

Release-ToDo-Analyse für Talvori

Priorisierte Aufgabenliste für den schnellstmöglichen Marktstart. Stand: 30.05.2026. Erstellt aus der Code-/Projektanalyse, ohne App-Code zu ändern.

Gesamtbewertung: Talvori ist technisch weit fortgeschritten, aber für einen öffentlichen Marktstart noch nicht release-ready. Der größte Engpass liegt nicht in der Grundarchitektur, sondern in Inhaltsqualität, vollständiger Lokalisierung, Lernsprach-Unterstützung und dem Entfernen sichtbarer Platzhalter/Debug-Flächen.

Quick Wins

- Sichtbare Platzhalter und Coming-soon-Elemente aus produktiven Pfaden entfernen oder klar ausblenden.
- Hartcodierte deutsche UI-Texte in einer Liste sammeln und die wichtigsten Screens sprachlich vereinheitlichen.
- Debug-/Developer-Zugänge vor Release absichern und nur in Debug-Builds erreichbar lassen.
- App-Metadaten wie pubspec-Beschreibung, Store-Texte, Datenschutz/AGB/Hilfe/Feedback aktualisieren.
- Alte Supabase-/VocabSort-Pfade als Legacy markieren und aus Nutzerpfaden heraushalten.

Kritische Blocker

- Vokabelqualität: Alle Wörter müssen einzeln geprüft werden, inklusive Übersetzung, Bedeutung, Dubletten, Kategorien und problematischen Einträgen.
- Lokalisierung: Deutsch, Englisch, Spanisch und Französisch brauchen vollständige, natürliche UI-Texte ohne Sprachmix.
- Spanisch/Französisch als Lerninhalte: Die App darf für den Marktstart nicht nur auf englisch-deutsche Inhalte ausgerichtet sein, wenn EN-Nutzer Spanisch oder Französisch lernen sollen.
- Lernsprache vs. App-Sprache: Datenmodell, UI und Importpfade müssen sauber zwischen Muttersprache/App-Sprache und Lernsprache unterscheiden.
- Sichtbare unfertige Bereiche: Platzhalter, Preview-Screens, Test-Wörter und leere Buttons müssen vor Release behandelt werden.

1. Muss vor Release erledigt werden

1. Vokabelbestand vollständig qualitätssichern

Bereich	Lerninhalte / Datenqualität
Warum wichtig	Direkt sichtbarer Kernnutzen. Fehlerhafte Übersetzungen oder Dubletten beschädigen Vertrauen sofort.
Priorität / Aufwand / Risiko	Muss / groß / hoch
Abhängigkeiten	Export/Review-Prozess, fachliche Sprachprüfung
Betroffene Dateien/Ordner	assets/local_import, docs/word-review, Supabase words tables, lib/core/local_database/seed
Umsetzungsidee	Audit-Tabelle erzeugen, Wort für Wort prüfen, Dubletten/Varianten markieren, problematische Einträge korrigieren oder vor Release ausblenden.
Reihenfolge / Marker	1. - Blocker, Datenqualität, Tests

2. App-Sprache vollständig lokalisieren

Bereich	i18n/I10n
Warum wichtig	Die App enthält viele hartcodierte deutsche Texte und muss pro Sprache konsistent wirken.
Priorität / Aufwand / Risiko	Muss / groß / hoch
Abhängigkeiten	Entscheidung für Flutter-I10n/ARB oder bestehende Lokalisierungsstrategie
Betroffene Dateien/Ordner	lib/features, lib/main.dart, pubspec.yaml, I10n/arb falls neu
Umsetzungsidee	Zentrale I10n-Struktur einführen, harte Strings priorisiert migrieren, alle Hauptscreens auf DE/EN/ES/FR prüfen.
Reihenfolge / Marker	2. - Blocker, Lokalisierung, UI/UX

3. Lernsprache und Muttersprache sauber trennen

Bereich	Datenmodell / UI / Import
Warum wichtig	Spanisch/Französisch für englische Nutzer ist ohne klare Sprachtrennung nicht marktfähig.
Priorität / Aufwand / Risiko	Muss / groß / hoch
Abhängigkeiten	Vokabeldatenmodell, Import- und Practice-Pfade
Betroffene Dateien/Ordner	lib/core/local_database, lib/features/home/application, lib/features/words, lib/features/import
Umsetzungsidee	Source/target/native/app language überall prüfen, harte en/de Annahmen entfernen, Provider und UI auf Sprachpaar ausrichten.
Reihenfolge / Marker	3. - Blocker, Neue Inhalte, Lokalisierung

4. Spanische und französische Lerninhalte aufbauen

Bereich	Lerninhalte
Warum wichtig	Zielgruppe soll Spanisch/Französisch lernen können; dafür fehlen vollständige Inhalte.
Priorität / Aufwand / Risiko	Muss / groß / hoch
Abhängigkeiten	Sprachtrennung, Qualitätsworkflow
Betroffene Dateien/Ordner	assets/local_import, Supabase word sources, docs/word-review
Umsetzungsidee	EN->ES und EN->FR Wortwelten analog zur englischen Basis erstellen, Top-Wörter und Alltagsthemen vollständig abdecken.
Reihenfolge / Marker	4. - Blocker, Neue Inhalte, Datenqualität

5. Produktive Platzhalter entfernen oder verstecken

Bereich	UI/UX
Warum wichtig	Preview-/Dummy-Elemente wirken unfertig und erzeugen Support-Frust.
Priorität / Aufwand / Risiko	Muss / mittel / mittel
Abhängigkeiten	Release-Scope-Entscheidung
Betroffene Dateien/Ordner	lib/features/home/ui/screens/settings_screen.dart, profile_screen.dart, vocabulary_level_test_screen.dart, word_duel_preview_screen.dart, chat_detail_screen.dart, course_screen.dart
Umsetzungsidee	Alle Coming-soon, Dummy, Test und leeren Buttons erfassen; vor Release entweder fertigstellen, ausblenden oder klar als nicht verfügbar kennzeichnen.
Reihenfolge / Marker	5. - Blocker, Quick Win, UI/UX

6. Debug- und Developer-Bereiche release-sicher machen

Bereich	Release-Sicherheit
Warum wichtig	Import- oder Debug-Funktionen dürfen nicht versehentlich für Nutzer sichtbar sein.
Priorität / Aufwand / Risiko	Muss / klein / mittel
Abhängigkeiten	Build-Konfiguration
Betroffene Dateien/Ordner	lib/main.dart, lib/features/home/ui/screens/settings_screen.dart, debug routes
Umsetzungsidee	kDebugMode-Gates prüfen, Release-Build testen, Debug-Routen und Supabase-Importzugänge absichern.
Reihenfolge / Marker	6. - Blocker, Quick Win

7. Rechtliches und Store-Basics ergänzen

Bereich	Release / Store
Warum wichtig	Datenschutz, AGB, Hilfe, Feedback und Bewertung sind im Marktstart sichtbar oder erwartbar.
Priorität / Aufwand / Risiko	Muss / mittel / mittel
Abhängigkeiten	Rechtstexte / Store-Konzept
Betroffene Dateien/Ordner	lib/features/home/ui/screens/settings_screen.dart, pubspec.yaml, app store metadata
Umsetzungsidee	Platzhalter durch echte Links/Seiten ersetzen, pubspec-Beschreibung und Produkttexte release-tauglich machen.
Reihenfolge / Marker	7. - Blocker, Quick Win, UI/UX

2. Sollte vor Release erledigt werden

8. Wörter-prüfen-Flow auf Gerät final feinprüfen

Bereich	Wörter prüfen
Warum wichtig	Der Flow ist zentral für lokale Known-/Reviewed-Status und wurde stark iteriert.
Priorität / Aufwand / Risiko	Sollte / mittel / mittel
Abhängigkeiten	Testdaten mit realistischen Wortmengen
Betroffene Dateien/Ordner	lib/features/words/application/local_known_review_controller.dart, local_known_review_screen.dart, word_repository.dart
Umsetzungsidee	Mit echten Kategorien Wheel, Center-Sync, Last-Word, Reset, Counter, Filterlisten und Undo durchtesten.
Reihenfolge / Marker	8. - UI/UX, Tests

9. Tali/Vori getrennte Chatverläufe final verifizieren

Bereich	Companion / Chat
Warum wichtig	Nutzer erwarten getrennte Identität, Name, Avatar und Verlauf.
Priorität / Aufwand / Risiko	Sollte / mittel / mittel
Abhängigkeiten	Mascot-Style-Auswahl
Betroffene Dateien/Ordner	lib/features/companion, lib/features/chat, lib/core/assets/talvori_mascot_assets.dart
Umsetzungsidee	Chat-ID, Avatar, Verlaufsspeicher und UI-Name je Figur prüfen und Tests ergänzen.
Reihenfolge / Marker	9. - UI/UX, Tests

10. Supabase-Auto-Sync-Strategie releasefähig entscheiden

Bereich	Daten / Backend
Warum wichtig	Automatisches Nachladen lokaler Wörter kann unkontrolliert wirken und ist releasekritisch.
Priorität / Aufwand / Risiko	Sollte / mittel / hoch
Abhängigkeiten	Produktentscheidung Offline/Online-Seed
Betroffene Dateien/Ordner	lib/main.dart, lib/core/sync, lib/core/local_database
Umsetzungsidee	Klare Regel: gebündelte Seed-Daten vs. kontrollierte Syncs, Fehlerzustände und Offline-Fallbacks definieren.
Reihenfolge / Marker	10. - Datenqualität, Tests

11. Wortspiele Release-Scope festlegen

Bereich	Wortspiele
Warum wichtig	Einige Spiele sind vorbereitet oder KI-/Supabase-abhängig.
Priorität / Aufwand / Risiko	Sollte / mittel / mittel
Abhängigkeiten	Content- und Backend-Reife
Betroffene Dateien/Ordner	lib/features/games, lib/features/home/application/vocab_controller.dart
Umsetzungsidee	Nur stabile Spiele im Release sichtbar lassen; Preview-Modi und vorbereitete Stati aus Nutzerpfaden entfernen.
Reihenfolge / Marker	11. - UI/UX, Tests

12. Onboarding und erste Nutzerreise prüfen

Bereich	Onboarding / Home
Warum wichtig	Ein Marktstart braucht eine verständliche erste Minute.
Priorität / Aufwand / Risiko	Sollte / mittel / mittel
Abhängigkeiten	Release-Scope, Sprachwahl
Betroffene Dateien/Ordner	lib/main.dart, lib/features/onboarding, lib/features/home
Umsetzungsidee	Onboarding-Gate, Sprache, Lernziel, erste Wortwelt und erste Übung auf Klarheit prüfen.
Reihenfolge / Marker	12. - UI/UX, Lokalisierung

13. Testabdeckung für kritische Release-Pfade ergänzen

Bereich	Tests
Warum wichtig	Viele komplexe lokale Zustände brauchen Regressionstests.
Priorität / Aufwand / Risiko	Sollte / mittel / mittel
Abhängigkeiten	Release-Scope stabil
Betroffene Dateien/Ordner	test/features, test/core/local_database
Umsetzungsidee	Smoke-Tests für Onboarding, Home, Wörter prüfen, Filterlisten, Practice, Settings, Sprache und Import ergänzen.
Reihenfolge / Marker	13. - Tests

3. Kann nach Release kommen

14. Alte Supabase-/VocabSort-UI vollständig ablösen

Bereich	Legacy / Wörter prüfen
Warum wichtig	Alte Pfade bleiben Wartungsrisiko, sind aber nicht zwingend blockierend, wenn nicht erreichbar.
Priorität / Aufwand / Risiko	Später / mittel / mittel
Abhängigkeiten	Neue lokale Review-UI stabil
Betroffene Dateien/Ordner	lib/features/words/ui/screens/vocab_sort_screen.dart, vocab_sort_controller.dart
Umsetzungsidee	Legacy-Pfad entfernen oder lokal refactoren, sobald V-Button-Flow final stabil ist.
Reihenfolge / Marker	14. - Tech Debt

15. Home-Glow pro Screen statt pauschal prüfen

Bereich	UI/UX
Warum wichtig	Visuelle Qualität ist wichtig, darf aber keine Logik blockieren.
Priorität / Aufwand / Risiko	Später / klein / niedrig
Abhängigkeiten	Release-Screens stabil
Betroffene Dateien/Ordner	lib/features/home, lib/features/words, lib/features/tagesimpuls
Umsetzungsidee	Nur sichtbare harte Clipping-Probleme je Screen isoliert korrigieren.
Reihenfolge / Marker	15. - UI/UX, Quick Win

16. Erklärung von Kenn ich / Noch lernen / Pausiert verbessern

Bereich	UX / Wörter
Warum wichtig	Statusbegriffe können Nutzer verwirren.
Priorität / Aufwand / Risiko	Später / klein / niedrig
Abhängigkeiten	Finale Statuslogik
Betroffene Dateien/Ordner	local_word_list_screen.dart, local_known_review_screen.dart
Umsetzungsidee	Kurze Mikrotex te oder Hilfezustände ergänzen, ohne Flow zu überladen.
Reihenfolge / Marker	16. - UI/UX

17. Favoriten, Meine Wörter und eigener Mix konsolidieren

Bereich	Wortlisten
Warum wichtig	Ähnliche Konzepte können unklar wirken.
Priorität / Aufwand / Risiko	Später / mittel / mittel
Abhängigkeiten	Lokale Source-Definition final
Betroffene Dateien/Ordner	local_learning_source.dart, local_words_for_source_provider.dart, local_word_list_screen.dart
Umsetzungsidee	Quellen semantisch schärfen, Labels prüfen, leere Zustände verbessern.
Reihenfolge / Marker	17. - UI/UX

4. Größere spätere Ausbaustufen

18. Mehrsprachige Content-Pipeline aufbauen

Bereich	Content Operations
Warum wichtig	Für mehrere Ziel-/Lernsprachen braucht Talvori einen dauerhaften Prüf- und Importprozess.
Priorität / Aufwand / Risiko	Ausbaustufe / groß / hoch
Abhängigkeiten	L10n und Sprachmodell stabil
Betroffene Dateien/Ordner	tools/scripts, docs/word-review, assets/local_import, Supabase data model
Umsetzungsidee	Pipeline für Quellen, Übersetzung, QA, Meaning-Variants, Dubletten und Release-Exports bauen.
Reihenfolge / Marker	18. - Neue Inhalte, Datenqualität

19. KI-Companion produktionsreif machen

Bereich	Companion / KI
Warum wichtig	Langfristig wichtig für Differenzierung, aber nicht zwingend für einen schlanken MVP.
Priorität / Aufwand / Risiko	Ausbaustufe / groß / hoch
Abhängigkeiten	Kosten, Datenschutz, Moderation, Chatidentitäten
Betroffene Dateien/Ordner	lib/features/companion, lib/features/chat, lib/features/ai
Umsetzungsidee	Tali/Vori-Personas, getrennte Verläufe, Safety, Offline-/Fehlerzustände und Kostenlimits definieren.
Reihenfolge / Marker	19. - UI/UX, Tests

20. Store-Analytics und Release-Monitoring einführen

Bereich	Release Operations
Warum wichtig	Für schnelles Lernen nach Release nötig.
Priorität / Aufwand / Risiko	Ausbaustufe / mittel / mittel
Abhängigkeiten	Datenschutzkonzept
Betroffene Dateien/Ordner	analytics/instrumentation future areas
Umsetzungsidee	Privacy-schonende Events für Onboarding, Practice, Import, Retention und Fehlerzustände planen.
Reihenfolge / Marker	20. - Tests

Empfohlene Arbeitsreihenfolge

- Release-Scope einfrieren: Welche Screens, Spiele und KI-Flows sind im MVP sichtbar?
- Debug-/Platzhalter-Flächen aus produktiven Pfaden entfernen.
- Sprachtrennung und Lokalisierungsstruktur entscheiden und einführen.
- Vokabel-QA für bestehenden Bestand durchführen und kritische Daten bereinigen.
- Spanisch- und Französisch-Datenbasis aufbauen oder Release-Ziel entsprechend einschränken.
- Wörter-prüfen, Filterlisten, Practice und Onboarding auf echten Geräten testen.
- Store-, Datenschutz-, Hilfe- und Feedback-Basics ergänzen.
- Finale Regression: flutter test, Release-Build, Smoke-Test auf Gerät.

Offene Fragen

- Soll der erste Marktstart wirklich Spanisch und Französisch enthalten, oder wird der MVP bewusst kleiner gestartet?
- Welche Muttersprache ist initial primär: Deutsch, Englisch oder beide gleichwertig?
- Welche Wortspiele sind für Release wirklich freigegeben?
- Soll Supabase beim ersten Start automatisch lokale Wörter nachladen, oder werden geprüfte Seeds mitgeliefert?
- Welche KI-/Companion-Funktionen sind Teil des MVP und welche bleiben später?

Analyse-Hinweise

- Keine App-Dateien wurden für diese PDF-Erstellung verändert.
- Bekannter Arbeitsbaum-Hinweis: Vor dieser PDF existierte bereits eine uncommitted Änderung an `lib/features/words/ui/cards/word_card.dart` aus dem Browser-Fenster-Designblock.
- Die PDF selbst ist ein neues Dokument in `docs/release_todo_analysis.pdf`.